Andrés Huertas – 20242560
Juan David Colorado – 202415548

Alfabeto de Objectilang

$$g =< N, \Sigma, S_0, P >$$

$$S_0 = \{Modulr\}$$

$$N = \{\ Module,\ MethodAssign,\ MainMethod,\ ObjectDef,\ GenericParams,\ GenericParam,\ ConstructorParamList,\ ConstructorParam,\ Block,\ MethodDef,\ MethodParamList,\ MethodParam,\ OverrideDef,\ ClassBody,\ Stmt,\ ValDecl,\ VarDecl,\ Assignment,\ ForStmt,\ ForBinding,\ Pattern,\ Expr,\ RangeExpr,\ IfExpr,\ OrExpr,\ AndExpr,\ InExpr,\ CompExpr,\ AddExpr,\ MulExpr,\ PowExpr,\ UnaryExpr,\ Postfix,\ PostfixOp,\ Primary,\ NewExpr,\ ArgList,\ Type,\ Identifier,TypeIdentifier\ Literal,\ IntLiteral,\ BoolLiteral,\ StringLiteral,\ StringInterpolation,\ StringChar,\ EscapeChar,\ Comment,\ CommentChar,\ Space,\ Tab,\ Newline,\ PunctuationChar,\ Digit,\ Letter,\ LowercaseLetter,\ UppercaseLetter\ \}$$

$\Sigma = \{$ a … z, A … Z, 0 … 9, space, textbackslasht, textbackslashn, textbackslashr, !, ", #, $, %, &, ′, (,), ∗, +, ,, −, ., /, :, ;, <, =, >, ?, @, [, textbackslash,], ^{}, _, `, {, |, }, ==$\}$[1]

P

Module::= (ObjectDef | MethodAssign)* MainMethod

MethodAssign ::= TypeIdentifier "." Identifier "=" MethodDef

MainMethod ::= "method" "main" Block

ObjectDef ::= "object" TypeIdentifier (GenericParams)? ("extends" Type)? ("(" (ConstructorParamList)? ")")?

GenericParams ::= "[" GenericParam ("," GenericParam)* "]"
GenericParam ::= ("+" | "-")? TypeIdentifier

[1] Para el alfabeto, se utilizo inteligencia artificial para poder utilizer LATEX. Sin embargo, dado que LATEX pone problema para algunos de los caracteres utilizados en el lenguaje, se cambio la tipografia para que se entendiera mejor.

Andrés Huertas – 20242560
Juan David Colorado – 202415548

ConstructorParamList ::= ConstructorParam ("," ConstructorParam)*
ConstructorParam ::= ("val" | "var") Identifier ":" Type

Block ::= "{" (Newline)* (Stmt (Newline)+)* (Stmt)? (Newline)* "}"

MethodDef ::= "method" "(" (MethodParamList)? ")" (":" Type)? (Block)?
MethodParamList ::= MethodParam ("," MethodParam)*
MethodParam ::= Identifier ":" Type

OverrideDef ::= "override" Identifier "(" (MethodParamList)? ")" ":" Type "=" Block
ClassBody ::= "{" (OverrideDef)+ "}"

Stmt ::= ValDecl
    | VarDecl
    | Assignment
    | ForStmt
    | Expr

ValDecl ::= "val" Identifier (":" Type)? "=" Expr
VarDecl ::= "var" Identifier (":" Type)? "=" Expr
[2]Assignment ::= Postfix "=" Expr

ForStmt ::= "for" "(" ForBinding ")" Stmt
ForBinding ::= Pattern ("<-" | "in") Expr
Pattern ::= Identifier | "(" Identifier ("," Identifier)+ ")"

Expr ::= IfExpr
    | RangeExpr

RangeExpr ::= OrExpr ("->" OrExpr)?
IfExpr ::= "if" Expr "then" Expr "else" Expr

OrExpr ::= AndExpr ("or" AndExpr)*
AndExpr ::= InExpr ("and" InExpr)*

---

[2]La regla Assignment ::= Postfix "=" Expr permite sintácticamente que el lado izquierdo sea cualquier Postfix (incluyendo llamadas a método o instanciaciones, ej. foo() = 5). Restringir esto formalmente en la gramática requeriría introducir un no-terminal LValue que duplicaría gran parte de la lógica de Postfix, sin eliminar del todo casos ambiguos (ej. indexación).

Andrés Huertas – 20242560
Juan David Colorado – 202415548

```
InExpr ::= CompExpr ( "in" CompExpr )?
CompExpr ::= AddExpr ( ( "<" | ">" | "<=" | ">=" | "<>" | "==" ) AddExpr )?
AddExpr ::= MulExpr ( ( "+" | "-" ) MulExpr )*
MulExpr ::= PowExpr ( ( "*" | "/" | "%" ) PowExpr )*
PowExpr ::= UnaryExpr ( "**" PowExpr )?

UnaryExpr ::= ( "neg" | "-" )* Postfix

Postfix ::= Primary ( PostfixOp )*
PostfixOp ::= "." Identifier ( "(" ( ArgList )? ")" )?
        | "(" ( ArgList )? ")"

Primary ::= Literal
        | Identifier
        | TypeIdentifier
        | NewExpr
        | "(" Expr ")"

NewExpr ::= "new" Type "(" ( ArgList )? ")" ( ClassBody )? | "new" Type ClassBody
ArgList ::= Expr ( "," Expr )*

Type ::= TypeIdentifier ( "[" Type ( "," Type )* "]" )?

Identifier ::= LowercaseLetter (Letter | “_“ | Digit )*

TypeIdentifier ::= UppercaseLetter (Letter | “_“ | Digit )*

Literal ::= StringLiteral | IntLiteral | BoolLiteral

BoolLiteral ::= "true" | "false"
IntLiteral ::= ( Digit )+

StringLiteral ::= "\"" ( StringChar | StringInterpolation )* "\""[3]
StringInterpolation ::= "${" Expr "}" | "$" Identifier

StringChar ::= Letter | Digit | Space | PunctuationChar | EscapeChar
EscapeChar ::= "\\" "\"" | "\\" "$" | "\\" "\\" | "\\" "n" | "\\" "t" | "\\" "r"
```

[3] Los “\” se utilizan para indicar donde empieza la Comilla y donde acaba. Es netamente de presentación.

Andrés Huertas – 20242560
Juan David Colorado – 202415548

Comment ::= "//" (CommentChar)* (Newline)?
CommentChar ::= Letter | Digit | Space | Tab | PunctuationChar | "\\" | "\"" | "$"

Space ::= " "
Tab ::= "\t"
Newline ::= "\n" | "\r"

PunctuationChar ::= "!" | "#" | "%" | "&" | "'" | "(" | ")" | "*" | "+" | "," | "-" | "." | "/" | ":" | ";" | "<" | "=" | ">" | "?" | "@" | "[" | "]" | "^" | "_" | "`" | "{" | "|" | "}"

Digit ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Letter ::= LowercaseLetter | UppercaseLetter

LowercaseLetter ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z"
UppercaseLetter ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"

Cambios para la entrega 2 (subrayados en amarillo)
Primeramente, el estado inicial del lenguaje ahora va a ser Module y no Program. Segundo, tuvimos un problema con Identifier porque entonces no permite diferenciar entre cuando tenemos el nombre de una variable y el nombre de un tipo de objeto. En casos de variables, el identificador debería empezar con minúscula o con “-“, mientras que para los tipos de objetos se deben utilizar mayúsculas. Dicho eso, agregamos un TypeIdentifier que nos permitiera aclarar cuando los tipos son para estructuras, entidades abstractas o tipos de datos para el sistema. Identifier paso a solo tener una letra minúscula al principio que se utilizara para los nombres de elementos ejecutables, instancias y variables del programa. Otro aspecto a tener en cuenta es que una expresión x = 5 iba generar ambigüedad previamente, entonces agregamos al alfabeto el “==” para poder diferenciar entre asignación y comparación lo cual permite que una instrucción x ==5 y x =5 tenga cada una un único árbol de derivación.

Por último, se ajustó la producción Primary eliminando la alternativa Block, ya que permitía utilizar bloques en cualquier contexto donde se aceptara una expresión, incluyendo condiciones. De esta manera, los bloques quedan restringidos a las producciones que los requieren explícitamente, como MainMethod, MethodDef y OverrideDef, sin modificar su definición.

Andrés Huertas – 20242560
Juan David Colorado – 202415548